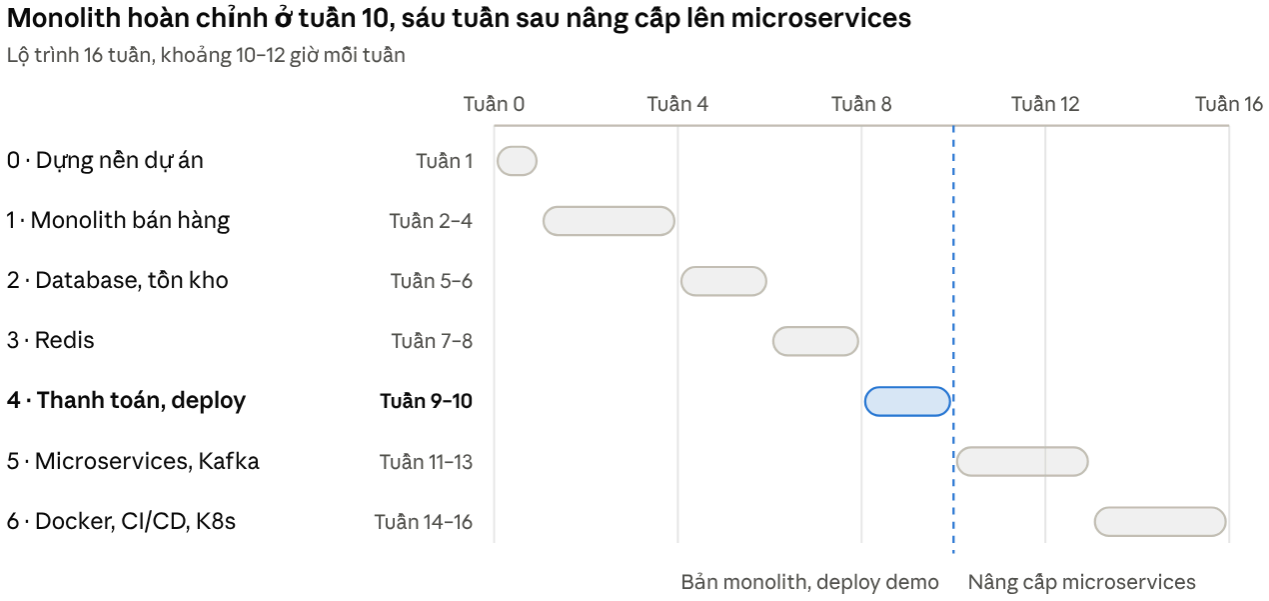
Lộ trình lên Java Middle qua dự án web bán pate cho mèo

Oct 6, 2026 · @Linh Nguyễn

Trong 16 tuần, với khoảng 10–12 giờ mỗi tuần, bạn xây một web bán pate tươi cho mèo từ monolith đơn giản đến microservices với Kafka, Redis và Kubernetes. Đây là dự án để học, nhưng nghiệp vụ lấy đúng từ cửa hàng pate bạn từng tự vận hành: chốt đơn 18:00, tối đa 50 hộp mỗi ngày.

Tổng quan



lộ trình 16 tuần · 7 giai đoạn

Bốn giai đoạn đầu xây bản monolith hoàn chỉnh và deploy bản demo vào cuối tuần 10; hai giai đoạn sau trên nhánh riêng, dành cho Kafka, microservices và Kubernetes.

Vì sao chọn web bán pate cho mèo

Đây là ý tưởng tốt hơn hầu hết dự án mẫu, vì nghiệp vụ là thật: bạn từng tự kinh doanh pate nên hiểu khách đặt thế nào, chốt đơn ra sao, vướng ở đâu. Khi phỏng vấn, "tôi xây hệ thống dựa trên quy trình cửa hàng tôi từng tự vận hành" thuyết phục hơn nhiều so với một bản sao Shopee.

Mô hình đặt trước, chốt đơn rồi mới nấu và giao của bạn có những bài toán kỹ thuật mà dự án bán hàng thông thường không có:

- **Làm theo đơn:** chốt đơn lúc 18:00 mỗi ngày và nhận tối đa 50 hộp, vì nấu bằng tay; đủ suất thì khóa đặt hàng cho ngày đó.
- **Kế hoạch nấu:** gom đơn của một ngày thành số hũ cần nấu theo từng vị, rồi tính lượng nguyên liệu cần mua theo công thức.
- **Nguyên liệu tươi có hạn dùng:** tồn kho là thịt, cá nguyên liệu chứ không phải hũ pate làm sẵn; nhập theo lô, dùng lô gần hết hạn trước (FIFO).
- **Giao sau khi nấu:** đơn đi qua các bước đặt, xác nhận, nấu, đóng gói, giao; khách chọn khung giờ nhận.
- **Tranh suất cuối:** ngày nấu còn vài suất mà nhiều người đặt cùng lúc, phải chống nhận quá công suất.
- **Khách mua lặp lại:** gói đăng ký pate mỗi tuần, tự tạo đơn cho ngày nấu kế tiếp.

Những bài toán này dẫn tự nhiên đến transaction, locking, Redis, Kafka và job định kỳ, đúng những thứ nhà tuyển dụng Middle hỏi.

Ý tưởng thay thế, nếu sau này bạn muốn làm dự án thứ hai: hệ thống đặt lịch spa và tắm cho thú cưng (bài toán trùng lịch, khóa khung giờ), hoặc đặt vé sự kiện (bài toán nhiều người tranh một chỗ). Nhưng với bạn, cửa hàng pate vẫn là lựa chọn số một vì bạn hiểu nghiệp vụ từ chính kinh nghiệm kinh doanh.

Phạm vi sản phẩm

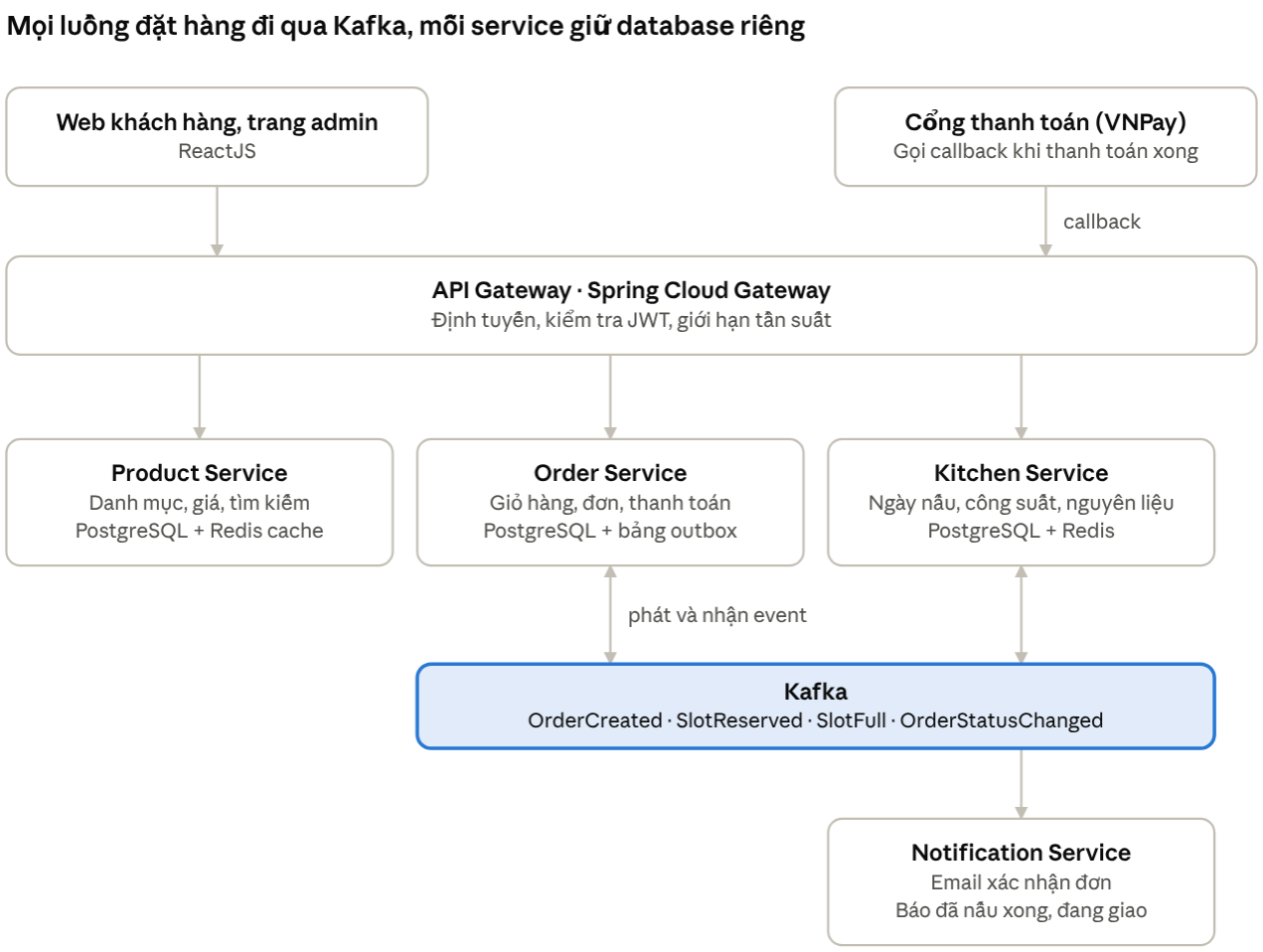
Các tính năng cốt lõi hoàn thành trong bản monolith ở giai đoạn 4; phần nâng cao và microservices là bước nâng cấp sau đó. Khi phỏng vấn vẫn nên nói rõ: với quy mô 50 hộp mỗi ngày, monolith là đủ, microservices ở đây là để học.

Phân hệ	Tính năng	Dùng cho
Khách hàng	Đăng ký, đăng nhập, hồ sơ mèo (cân nặng, dị ứng), sổ địa chỉ	Cốt lõi
Sản phẩm	Danh mục, vị (gà, cá hồi, bò), quy cách hũ, giá, ảnh, tìm kiếm, lọc	Cốt lõi
Đơn hàng	Đặt trước theo ngày nấu, giờ chốt đơn, chọn khung giờ giao, trạng thái đơn, hủy đơn	Cốt lõi
Sản xuất	Lịch ngày nấu, giới hạn số hũ mỗi ngày, kế hoạch nấu, nguyên liệu theo lô và hạn dùng	Cốt lõi
Thanh toán	COD, chuyển khoản QR, cổng thanh toán (VNPay sandbox)	Cốt lõi

Phân hệ	Tính năng	Dùng cho
Quản trị	Quản lý sản phẩm, đơn, kế hoạch nấu, nguyên liệu, báo cáo doanh thu theo ngày	Cốt lõi
Thông báo	Email xác nhận đơn, báo đơn đã nấu xong và đang giao, cảnh báo nguyên liệu sắp hết hạn	Cốt lõi
Gói định kỳ	Đăng ký giao pate mỗi tuần, tự tạo đơn	Nâng cao
Mở suất mới	Mở nhận đơn cho ngày nấu mới vào giờ cố định, nhiều người tranh suất	Để học
Microservices	Tách Order, Kitchen, Notification, giao tiếp qua Kafka	Để học

Frontend: không phải trọng tâm. Dùng ReactJS bạn đã biết để làm nhanh; nếu nhắm đến các công ty dùng Vue.js thì làm trang quản trị bằng Vue 3 để có thêm kinh nghiệm.

Kiến trúc đích



kiến trúc đích sau giai đoạn 5 · 4 service, 1 message broker

Đây là hình dạng cuối cùng của nhánh học sau giai đoạn 5: Order và Kitchen không gọi nhau trực tiếp mà trao đổi event qua Kafka. Bản monolith ở giai đoạn 1–4 gộp tất cả trong một ứng dụng và một database.

Giai đoạn 0 · Tuần 1: Dựng nền dự án

Mục tiêu: một project Spring Boot chạy được, có database, có cấu trúc sạch để lớn dần.

- [ ] Tạo repo GitHub, quy ước tên nhánh và commit (feature/..., fix/...)
- [ ] Spring Boot 3, Java 21, Maven hoặc Gradle
- [ ] PostgreSQL chạy bằng docker-compose
- [ ] Flyway để quản lý thay đổi schema
- [ ] Chia package theo nghiệp vụ: `product`, `order`, `inventory`, `customer` (mỗi package có controller, service, repository, dto)
- [ ] Cấu hình profile `dev` và `prod`, không để mật khẩu trong code

- [ ] Viết README: cách chạy dự án trên máy

Tự kiểm tra: vì sao chia package theo nghiệp vụ thay vì theo tầng (controller, service)? Vì sao dùng Flyway thay vì `ddl-auto=update`?

Giai đoạn 1 · Tuần 2–4: Monolith bán hàng cơ bản

Mục tiêu: khách xem sản phẩm, đặt hàng được; chủ shop quản lý được. Nắm chắc Spring và JPA.

- [ ] API sản phẩm: CRUD, phân trang, lọc theo vị và giá
- [ ] Đặt hàng theo ngày nấu; trạng thái đơn: Chờ xác nhận → Đã xác nhận → Đang nấu → Đã đóng gói → Đang giao → Hoàn thành / Đã hủy; chặn chuyển sai trạng thái
- [ ] Đăng ký, đăng nhập bằng Spring Security và JWT; phân quyền `CUSTOMER` và `ADMIN`
- [ ] Validation đầu vào (`@Valid`), xử lý lỗi tập trung bằng `@RestControllerAdvice`, mã lỗi thống nhất
- [ ] `@Transactional` cho luồng đặt hàng: tạo đơn và giữ suất nấu trong ngày cùng thành công hoặc cùng thất bại
- [ ] Tài liệu API bằng springdoc-openapi (Swagger UI)
- [ ] Unit test cho service bằng JUnit 5 và Mockito; integration test cho repository bằng Testcontainers
- [ ] Trang web đơn giản: danh sách sản phẩm, giỏ hàng, trang quản trị đơn

Tự kiểm tra: vì sao gọi một method `@Transactional` từ method khác trong cùng class thì transaction không chạy? Exception nào làm rollback, exception nào không? JWT hết hạn thì xử lý thế nào (refresh token)?

Giai đoạn 2 · Tuần 5–6: Database, công suất nấu và nguyên liệu

Mục tiêu: hiểu sâu database, phần bị hỏi nhiều nhất khi phỏng vấn Middle.

- [ ] Bảng `cooking_day`: ngày nấu, giờ chốt đơn (mặc định 18:00), công suất tối đa (mặc định 50 hộp, admin chỉnh được theo ngày), số hộp đã nhận
- [ ] Chống nhận quá công suất khi nhiều người đặt cùng lúc: thử cả optimistic locking (`@Version`) và pessimistic locking (`SELECT ... FOR UPDATE`), so sánh
- [ ] Viết test giả lập 1.000 người cùng đặt khi chỉ còn 3 hộp, chứng minh không bao giờ nhận quá 50 hộp; đơn đặt sau 18:00 tự chuyển sang ngày nấu kế tiếp
- [ ] Bảng `recipe` và `ingredient_lot`: công thức từng vị, nguyên liệu nhập theo lô có hạn dùng
- [ ] Kế hoạch nấu: từ đơn của một ngày, tính số hũ mỗi vị và lượng nguyên liệu cần; trừ nguyên liệu theo lô gần hết hạn trước (FIFO)

- [ ] Tạo dữ liệu giả khoảng 100.000 đơn, viết báo cáo doanh thu theo ngày, đo thời gian
- [ ] Đọc `EXPLAIN ANALYZE`, thêm index phù hợp, đo lại và ghi số liệu trước và sau
- [ ] Tìm và sửa lỗi N+1 khi lấy danh sách đơn kèm sản phẩm (fetch join, `@EntityGraph`)
- [ ] Viết lại truy vấn báo cáo và kế hoạch nấu bằng MyBatis để so sánh với JPA

Tự kiểm tra: khi nào index không được dùng? Thứ tự cột trong composite index quan trọng thế nào? Các mức isolation level khác nhau ra sao? Khi nào chọn optimistic, khi nào chọn pessimistic locking?

Giai đoạn 3 · Tuần 7–8: Redis

Mục tiêu: dùng Redis cho đúng ba việc hay gặp nhất: cache, giới hạn tần suất và khóa phân tán.

- [ ] Thêm Redis vào docker-compose
- [ ] Cache danh sách sản phẩm bằng Spring Cache (`@Cacheable`, `@CacheEvict`), xóa cache khi admin sửa giá; đo tốc độ trước và sau
- [ ] Giới hạn số lần gọi API đăng nhập và đặt hàng (rate limiting)
- [ ] Mở suất cho ngày nấu mới: trừ số suất còn lại trên Redis bằng thao tác nguyên tử (`DECR` hoặc Lua script), sau đó mới ghi xuống PostgreSQL
- [ ] Khóa phân tán bằng Redisson cho job chạy định kỳ, tránh hai instance cùng chạy
- [ ] Lưu danh sách token đã đăng xuất (blacklist JWT) với TTL

Tự kiểm tra: cache bị lệch với database thì xử lý thế nào? Cache penetration, cache breakdown, cache avalanche là gì và cách phòng? Redis chết thì hệ thống còn chạy không?

Giai đoạn 4 · Tuần 9–10: Thanh toán, thông báo, job định kỳ

Mục tiêu: tích hợp với hệ thống bên ngoài và hoàn thiện bản monolith.

- [ ] Tích hợp cổng thanh toán ở môi trường sandbox (ví dụ VNPay): tạo link thanh toán, nhận kết quả qua callback
- [ ] Xử lý callback idempotent: cổng gọi lại hai lần thì đơn không bị xác nhận hai lần; kiểm tra chữ ký
- [ ] Tự hủy đơn chưa thanh toán sau 15 phút và trả lại suất nấu
- [ ] Gửi email xác nhận đơn bất đồng bộ (`@Async`), không làm chậm API đặt hàng
- [ ] Job hằng ngày (`@Scheduled`): đến giờ chốt thì khóa đơn và xuất kế hoạch nấu, cảnh báo nguyên liệu sắp hết hạn, tự tạo đơn cho gói định kỳ
- [ ] Log có cấu trúc, gắn mã request vào mọi dòng log để truy vết một đơn hàng

- [ ] **Deploy bản demo lên một VPS nhỏ với tên miền và HTTPS, có dữ liệu mẫu và tài khoản dùng thử để nhà tuyển dụng bấm thử**

Tự kiểm tra: idempotency là gì và cách làm? Nếu đã trừ tiền nhưng server sập trước khi cập nhật đơn thì sao? `@Async` chạy trên thread pool nào, nên cấu hình ra sao?

Giai đoạn 5 · Tuần 11–13: Microservices và Kafka

Mục tiêu: tách hệ thống trên một nhánh riêng (`microservices`), hiểu cái được và cái mất khi tách. Bản monolith vẫn giữ ở nhánh main để so sánh.

- [ ] Tách 4 service: Product, Order, Kitchen (ngày nấu, công suất, nguyên liệu) và Notification; mỗi service một database riêng
- [ ] Spring Cloud Gateway làm cổng vào duy nhất, kiểm tra JWT tại gateway
- [ ] Kafka: Order phát `OrderCreated`, Kitchen giữ suất rồi phát `SlotReserved` hoặc `SlotFull`; khi đơn đổi trạng thái thì phát `OrderStatusChanged` để Notification báo khách
- [ ] Saga theo kiểu choreography: ngày nấu đã đủ suất thì đơn tự chuyển sang đã hủy
- [ ] Outbox pattern: ghi event vào bảng `outbox` trong cùng transaction với đơn, rồi mới đẩy lên Kafka
- [ ] Consumer idempotent: nhận trùng message không giữ suất hai lần; có retry và dead letter topic
- [ ] Resilience4j: retry, timeout và circuit breaker khi gọi REST giữa các service
- [ ] Viết một trang trong README so sánh monolith và microservices trên chính dự án này

Tự kiểm tra: partition và consumer group hoạt động thế nào? Thứ tự message được đảm bảo ở đâu? Vì sao không ghi database rồi gửi Kafka trực tiếp? Khi nào không nên dùng microservices?

Giai đoạn 6 · Tuần 14–16: Docker, CI/CD, Kubernetes, giám sát

Mục tiêu: đóng gói, tự động hóa và quan sát được hệ thống, ở mức một developer cần biết.

- [ ] Dockerfile nhiều stage cho mỗi service, image nhỏ, không chạy bằng root
- [ ] GitHub Actions: mỗi lần push thì build, chạy test, build image; merge vào `main` thì deploy bản monolith lên VPS
- [ ] Chạy bản microservices trên Kubernetes cục bộ (kind hoặc minikube): Deployment, Service, ConfigMap, Secret, Ingress
- [ ] Health check bằng Spring Actuator, cấu hình readiness và liveness probe
- [ ] Prometheus và Grafana: dashboard số đơn mỗi phút, thời gian phản hồi, tỷ lệ lỗi; một cảnh báo khi lỗi tăng

- [ ] Tự tạo một sự cố (tắt Inventory, làm chậm database), dùng log và dashboard tìm nguyên nhân, ghi lại như một báo cáo sự cố

Tự kiểm tra: Pod, Deployment, Service khác nhau thế nào? Readiness và liveness probe khác nhau ra sao? Khi API chậm bất thường trên production, bạn điều tra theo thứ tự nào?

Nhịp học mỗi tuần

Với lịch đi làm toàn thời gian, khoảng 10–12 giờ mỗi tuần là bền vững.

Khi nào	Thời lượng	Làm gì
3 tối trong tuần	1 giờ mỗi tối	Đọc lý thuyết của giai đoạn, trả lời các câu tự kiểm tra
Thứ Bảy	4 giờ	Code các đầu việc trong checklist
Chủ Nhật	3 giờ	Code tiếp, viết test, ghi chép

Cuối mỗi giai đoạn, làm đủ ba việc:

1. Viết vào README một mục "Quyết định thiết kế": vấn đề gì, chọn cách nào, vì sao, đánh đổi gì.
2. Ghi số liệu đo được (thời gian phản hồi, tốc độ truy vấn trước và sau).
3. Tự trả lời toàn bộ câu hỏi tự kiểm tra thành lời, như đang phỏng vấn.

Chậm hơn kế hoạch là bình thường; ưu tiên hiểu sâu từng giai đoạn hơn là làm cho xong. Giai đoạn 1 và 2 là nền tảng, đáng dành thêm thời gian nhất.

Đưa dự án vào CV và phỏng vấn

Không cần đợi xong 16 tuần. Sau giai đoạn 2 đã có thể đưa vào CV, và cập nhật dần theo tiến độ. Luôn để ở mục riêng **Personal Projects**, kèm link GitHub và link bản demo, không trộn vào kinh nghiệm ở FPT.

Ví dụ mục này sau khi hoàn thành (số liệu phải là số bạn đo được thật):

> **Fresh Cat Food Store** · Sole developer · github.com/... · [link demo]
>
> - Built an ordering system modeled on the make-to-order fresh cat food business I previously ran (18:00 cut-off, 50 jars per day).
> - Designed a make-to-order flow with daily cooking capacity, order cut-offs and ingredient planning; prevented over-booking under concurrent orders with optimistic locking.
> - Cut daily revenue report time from [X]s to [Y]ms with composite indexes on [100K] test orders.
> - Split the system into 4 services communicating via Kafka, using Saga, Outbox and idempotent consumers.
> - Tech: Java 21, Spring Boot 3, PostgreSQL, Redis, Kafka, Docker, Kubernetes, GitHub Actions, Grafana.

Khi phỏng vấn, dự án này cho bạn những câu chuyện có thật để kể: vấn đề nhận quá công suất nấu và cách giải, vì sao chọn optimistic locking, vì sao với quy mô 50 hộp mỗi ngày thì monolith là đủ, dù bạn đã làm microservices. Câu cuối này thường gây ấn tượng nhất, vì nó cho thấy bạn biết cân nhắc chi phí chứ không chạy theo công nghệ.